Ingeniería En Sistemas Y Redes Informáticas

Docente:

Ing. William Girón

Materia:

Programación IV

Nombre de la actividad:

Laboratorio 2

Estudiantes:

Karen Esmeralda Portillo Portillo SMSS202223

Yolanda Isabel Marroquín Ulloa SMSS047424

Fecha de entrega:

10 de abril del 2026

RESPONDER LAS SIGUIENTES PREGUNTAS:

1. **¿De qué forma manejaste el Login de usuarios? Explica con tus palabras porque en tu página funciona de esa forma.** El Login se los usuarios funcionan con el método POST el contenido predeterminado de los inputs porque, aunque las indicaciones se pedían un Login se nos menciono de manera presencial que en este no se debían ingresar datos.

   Luego, de los valores del input se comparan con lo registrado en la base de datos por medio de la consulta "SELECT id, nombre, password FROM usuarios WHERE nombre = ?" para ver si existe y hay coincidencias entonces se valida con la función password_verify comparando con la contraseña ingresada y si hay coincidencia se ingresa a la página.

2. **¿Por qué es necesario para las aplicaciones web utilizar bases de datos en lugar de variables?** Las variables en PHP como en todo programa independientemente de su lenguaje de programación tienen algo en común: su contenido desaparece cerrar el programa. En este caso, es al terminar la petición HTTP. Si guardáramos los datos de lugares turísticos en variables, toda la información se perdería cada vez que el servidor recibe una nueva solicitud.

   Una base de datos relacional (MySQL en este caso) proporciona persistencia real: los datos sobreviven entre sesiones, entre reinicios del servidor y entre diferentes usuarios simultáneos. Además, permite consultar, filtrar y ordenar la información de forma eficiente con SQL, garantizar la integridad referencial (como la relación entre lugares y usuarios), y escalar el sistema conforme crece la cantidad de registros sin afectar el rendimiento.

3. **¿En qué casos sería mejor utilizar bases de datos para su solución y en cuáles utilizar otro tipo de datos temporales como cookies o sesiones? Se recomienda usar base de datos cuando:**

   Los datos deben persistir permanentemente (registros de usuarios, lugares turísticos, historial).

Múltiples usuarios necesitan acceder y modificar la misma información.
Se requieren consultas complejas, filtros o reportes sobre grandes volúmenes de datos.

Normalmente las cookies y datos de sesión temporal son para cuando los datos son temporales y específicos de una sola visita del usuario (como el estado de login o preferencias de idioma).
Se quiere recordar preferencias menores del usuario entre visitas sin necesidad de registro (cookies persistentes).
Se necesita mantener un carrito de compras, filtros activos o datos de navegación durante una sesión activa.

4. **Describa brevemente sus tablas y los tipos de datos utilizados en cada campo; justifique la elección del tipo de dato para cada uno.**
   Tabla: usuarios
   - **id – INT AUTO_INCREMENT PRIMARY KEY:** Identificador único autoincremental. Se usa INT porque los IDs son siempre valores enteros positivos y se necesita eficiencia en las búsquedas por clave primaria.
   - **nombre – VARCHAR(100):** Almacena el nombre completo del usuario. VARCHAR es apropiado porque los nombres varían en longitud y no deben desperdiciar espacio fijo.
   - **email – VARCHAR(150) UNIQUE:** Correo electrónico con restricción de unicidad para evitar registros duplicados. VARCHAR permite longitudes variables típicas de un correo.
   - **password – VARCHAR(255):** Almacena el hash bcrypt generado por password_hash(). Se usa VARCHAR(255) porque el hash tiene longitud fija de 60 caracteres pero se deja margen para futuros algoritmos.
   - **created_at – TIMESTAMP DEFAULT CURRENT_TIMESTAMP:** Registra automáticamente la fecha y hora de creación. TIMESTAMP es eficiente para almacenar marcas de tiempo y permite comparaciones de fechas.

Tabla: lugares

- **id – INT AUTO_INCREMENT PRIMARY KEY:** Identificador único de cada lugar. INT es adecuado para claves primarias numéricas.
- **nombre – VARCHAR(150):** Nombre del lugar turístico. VARCHAR permite nombres de longitud variable sin desperdiciar espacio.
- **categoria – ENUM('natural','cultural','historico','religioso','recreativo'):** Se usa ENUM porque las categorías son un conjunto cerrado y predefinido de valores, garantizando integridad de datos a nivel de motor de base de datos.
- **descripcion – TEXT:** Para textos largos sin longitud máxima definida. TEXT es preferible a VARCHAR cuando el contenido puede ser extenso, como una descripción detallada de un lugar.
- **direccion – VARCHAR(255):** Dirección o referencia física del lugar. VARCHAR(255) cubre cualquier dirección razonablemente larga.
- municipio – VARCHAR(100): Nombre del municipio de San Miguel. VARCHAR es suficiente para nombres geográficos.
- **horario – VARCHAR(100):** Texto libre para describir el horario de visita (puede ser nulo). VARCHAR permite formatos variados como '8:00-17:00' o 'Solo fines de semana'.
- **entrada – DECIMAL(6,2) DEFAULT 0.00:** Precio de entrada en dólares. DECIMAL es obligatorio para valores monetarios porque evita los errores de redondeo que ocurren con FLOAT o DOUBLE.
- **calificacion – TINYINT CHECK(1-5):** Calificación del 1 al 5. TINYINT ocupa solo 1 byte, ideal para valores pequeños. El CHECK constraint garantiza que solo se almacenen valores válidos.
- **usuario_id – INT FOREIGN KEY:** Relaciona cada lugar con el usuario que lo registró. Al ser FK referenciando el id de usuarios (INT), ambos tipos deben coincidir para mantener la integridad referencial.
- **created_at – TIMESTAMP DEFAULT CURRENT_TIMESTAMP:** Fecha y hora de inserción del registro, generada automáticamente por el servidor de base de datos.